# JAVA COURSE

{ array }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.0

# INDICE

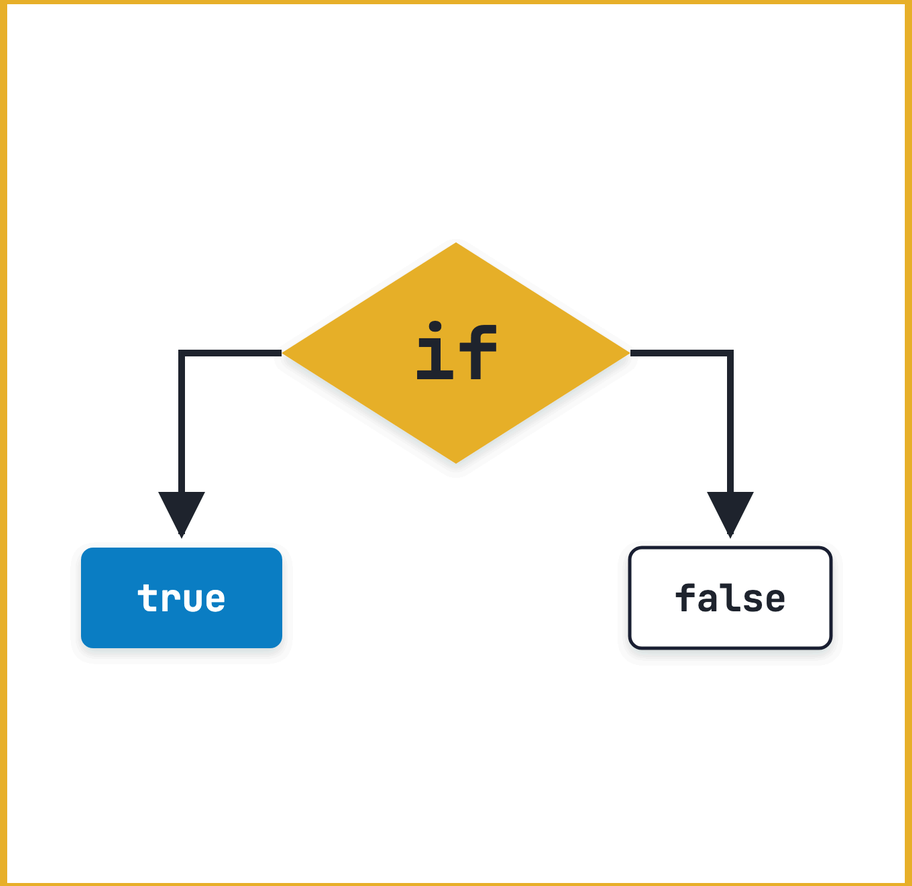

01 Cos'è un array

02 Iterare un array

03 Array e metodi

04 Array bidimensionali

# 01
# COS'È UN ARRAY



# IL PROBLEMA

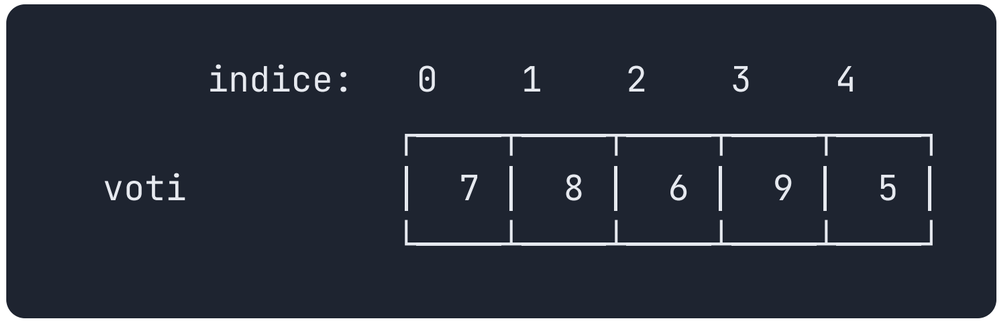
Per memorizzare cinque voti servirebbero cinque variabili separate. Se i voti diventano cento, il codice diventa ingestibile

```
int voto1 = 7;
int voto2 = 8;
int voto3 = 6;
int voto4 = 9;
int voto5 = 5;
// ... e se fossero 100?
```

- *Ogni variabile va dichiarata, inizializzata e gestita singolarmente*
- *Non si possono elaborare con un ciclo*
- *Serve una struttura che raggruppi valori dello stesso tipo*

# COS'È UN ARRAY

Un **array** è una sequenza di elementi dello stesso tipo, di **lunghezza fissa**, accessibili tramite un **indice numerico** che parte da `0`




- *Tutti gli elementi hanno lo **stesso tipo***
- *La dimensione viene decisa alla creazione e **non cambia***
- *Il primo elemento ha indice `0`, l'ultimo ha indice `lunghezza - 1`*

# DICHIARARE E CREARE

Due modi per creare un array in Java

```
// 1. specificare la dimensione (valori predefiniti)
int[] voti = new int[5];

// 2. elencare i valori tra graffe
int[] voti = {7, 8, 6, 9, 5};
```

- `int[]` *dichiara il tipo: "array di interi"*
- `new int[5]` *crea l'array con 5 celle, tutte a* `0`
- *Le graffe* `{...}` *creano e inizializzano in un solo passaggio*
- *Funziona con qualsiasi tipo:* `double[]`, `String[]`, `boolean[]`

# ACCEDERE E MODIFICARE

Si usa il nome dell'array seguito dall'indice tra parentesi quadre

```
int[] voti = {7, 8, 6, 9, 5};

// leggere un elemento
System.out.println(voti[0]);   // 7
System.out.println(voti[4]);   // 5

// modificare un elemento
voti[2] = 10;
System.out.println(voti[2]);   // 10
```

- *L'indice è sempre un valore intero (variabile o costante)*
- *Si può usare sia per leggere che per scrivere*

# LA LUNGHEZZA

Ogni array ha una proprietà `length` che restituisce il numero di elementi.

Attenzione: è una **proprietà**, non un metodo — **senza parentesi**

```
int[] voti = {7, 8, 6, 9, 5};
System.out.println(voti.length);    // 5

// ultimo elemento
System.out.println(voti[voti.length - 1]);  // 5
```

- `length` *senza* `()` *— a differenza di* `String.length()`
- *L'ultimo indice valido è sempre* `length - 1`

# INDICE FUORI DAI LIMITI

Se si accede a un indice negativo o maggiore/uguale a `length`, Java lancia un'eccezione di runtime

```
int[] voti = {7, 8, 6, 9, 5};
System.out.println(voti[5]);   // indice 5, ma length è 5!
```

Terminale

```
Exception in thread "main"
  java.lang.ArrayIndexOutOfBoundsExcept
  Index 5 out of bounds for length 5
```

- *Indici validi per un array di lunghezza 5:* `0, 1, 2, 3, 4`
- *È l'errore più comune quando si lavora con gli array*

# I VALORI PREDEFINITI

Quando si crea un array con `new`, ogni cella riceve un valore predefinito che dipende dal tipo

| Tipo | Valore predefinito |
|---|---|
| `int`, `long`, `short`, `byte` | `0` |
| `double`, `float` | `0.0` |
| `boolean` | `false` |
| `char` | `'\u0000'` |
| `String` (e altri oggetti) | `null` |

# ARRAY: PRIMA E DOPO

Il passaggio da variabili separate a un array

```java
// PRIMA: cinque variabili
int voto1 = 7;
int voto2 = 8;
int voto3 = 6;
int voto4 = 9;
int voto5 = 5;
int somma = voto1 + voto2 + voto3
          + voto4 + voto5;
```

```java
// DOPO: un array e un ciclo
int[] voti = {7, 8, 6, 9, 5};
int somma = 0;
for (int i = 0; i < voti.length; i++) {
    somma += voti[i];
}
```

- *Con l'array il codice funziona con 5 o 500 elementi senza cambiare nulla*

# QUIZ

Cosa stampa il seguente frammento?

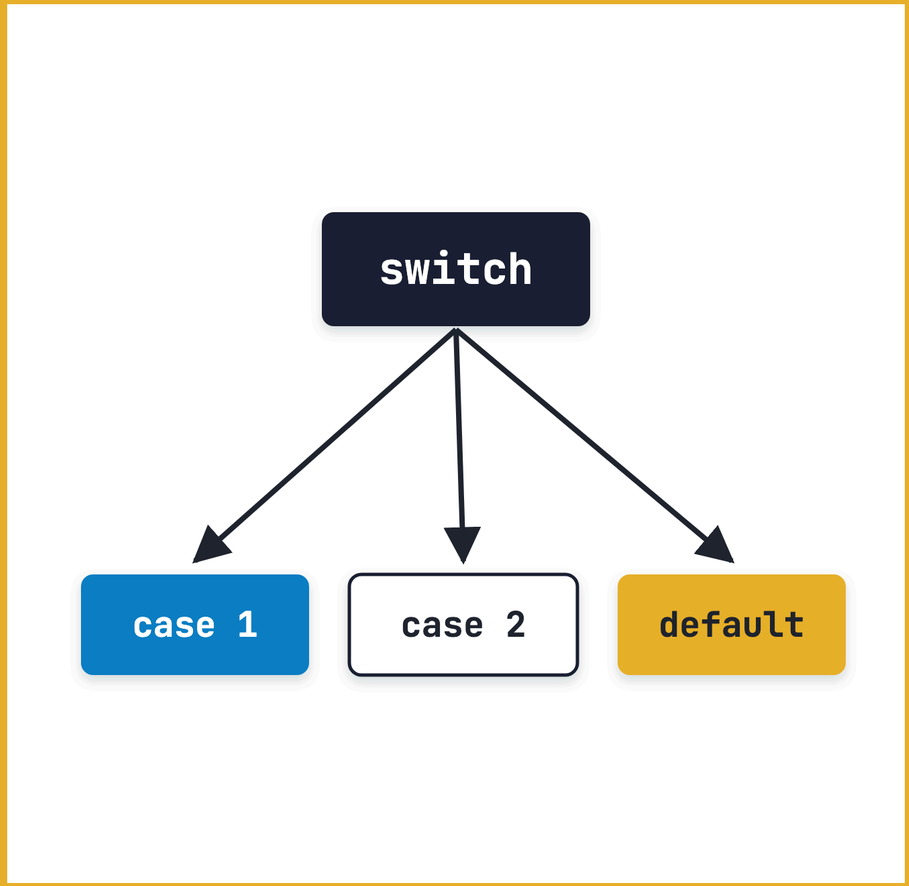
```
int[] numeri = new int[4];
numeri[1] = 10;
numeri[3] = 20;
System.out.println(numeri[0]);
System.out.println(numeri[1]);
System.out.println(numeri[2]);
System.out.println(numeri.length);
```

# RISPOSTE

```
int[] numeri = new int[4];
numeri[1] = 10;
numeri[3] = 20;
System.out.println(numeri[0]);    // 0     (predefinito)
System.out.println(numeri[1]);    // 10    (assegnato)
System.out.println(numeri[2]);    // 0     (predefinito)
System.out.println(numeri.length);// 4     (dimensione)
```

- *Le celle non assegnate contengono il valore predefinito*
- `length` *restituisce la dimensione, non l'ultimo indice*

# 02
# ITERARE UN ARRAY



# IL FOR CLASSICO

Il modo più comune per scorrere un array: un ciclo `for` con un indice che va da `0` a `length - 1`

```
int[] voti = {7, 8, 6, 9, 5};

for (int i = 0; i < voti.length; i++) {
    System.out.println("Voto " + i + ": " + voti[i]);
}
```

```
Terminale

Voto 0: 7
Voto 1: 8
Voto 2: 6
Voto 3: 9
Voto 4: 5
```

- `i < voti.length` — *mai* `<=`*, altrimenti si sfora*
- *L'indice* `i` *serve sia per la posizione che per il valore*

# IL FOR-EACH

Quando non serve l'indice, Java offre il **for-each**: più breve e meno soggetto a errori

```
int[] voti = {7, 8, 6, 9, 5};

for (int v : voti) {
    System.out.println(v);
}
```

- *Si legge: "per ogni* `int v` *contenuto in* `voti` *"*
- `v` *è una* ***copia*** *dell'elemento: modificarla non cambia l'array*
- *Non si può accedere all'indice corrente*
- *Per modificare gli elementi serve l'indice: si usa il for classico*

# QUANDO USARE QUALE

Una regola semplice per scegliere il ciclo giusto

| Situazione | Ciclo consigliato |
|---|---|
| Servono solo i valori | `for-each` |
| Serve l'indice (posizione) | `for` classico |
| Bisogna modificare gli elementi | `for` classico |
| Si scorre al contrario | `for` classico |
| Si scorrono due array insieme | `for` classico |

Nel dubbio, il `for` classico funziona sempre

# CONTATORI E ACCUMULATORI

Due schemi ricorrenti quando si lavora con gli array

```
// accumulatore: somma dei voti
int somma = 0;
for (int v : voti) {
    somma += v;
}
double media = (double) somma / voti.length;

// contatore: quanti sufficienti?
int sufficienti = 0;
for (int v : voti) {
    if (v >= 6) {
        sufficienti++;
    }
}
```

- *L'**accumulatore** raccoglie un totale (somma, prodotto)*
- *Il **contatore** conta quanti elementi soddisfano una condizione*

# IL MASSIMO E IL MINIMO

Per trovare il massimo si parte dal primo elemento e si confronta con tutti gli altri

```
int[] voti = {7, 8, 6, 9, 5};

int max = voti[0];
for (int i = 1; i < voti.length; i++) {
    if (voti[i] > max) {
        max = voti[i];
    }
}
System.out.println("Massimo: " + max);  // 9
```

- *Si inizializza* `max` *con* `voti[0]`*, non con* `0`
- *Per il minimo basta cambiare* `>` *in* `<`

# STAMPARE UN ARRAY

Attenzione: `System.out.println(voti)` non stampa i valori, ma un codice come `[I@1b6d3586`

```
int[] voti = {7, 8, 6, 9, 5};

// sbagliato
System.out.println(voti);
// → [I@1b6d3586

// corretto: con un ciclo
for (int v : voti) {
    System.out.print(v + " ");
}
// → 7 8 6 9 5
```

- *Nel prossimo capitolo vedremo* `Arrays.toString()`

# CHE COSA STAMPA?

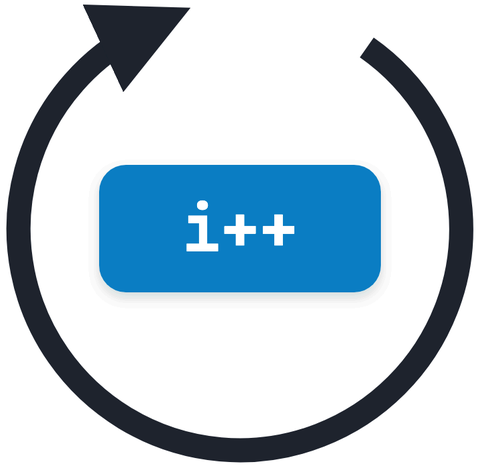
Analizzare il codice e prevedere l'output

```
int[] a = {3, 1, 4, 1, 5};
int r = 0;
for (int x : a) {
    if (x > 2) {
        r++;
    }
}
System.out.println(r);
```

# RISPOSTE

- *L'array contiene* `{3, 1, 4, 1, 5}`
- *Il contatore* `r` *si incrementa per ogni elemento maggiore di 2*
- *Elementi maggiori di 2:* `3`, `4`, `5` *— sono 3*
- *Il programma stampa* ***3***

# 03
# ARRAY E METODI



# PASSARE UN ARRAY A UN METODO

Il metodo riceve il **riferimento**: lavora sullo stesso array

```
static void raddoppia(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        arr[i] = arr[i] * 2;
    }
}
public static void main(String[] args) {
    int[] numeri = {1, 2, 3};
    raddoppia(numeri);
    // numeri ora è {2, 4, 6} — modificato!
}
```

- *Le modifiche dentro il metodo **si riflettono** sull'array originale*
- *Con un* `int` *invece il metodo riceve una copia del valore*

# RESTITUIRE UN ARRAY

Un metodo può creare e restituire un array. Il tipo di ritorno è `int[]`, `double[]`, ecc.

```
static int[] creaSequenza(int n) {
    int[] seq = new int[n];
    for (int i = 0; i < n; i++) {
        seq[i] = i + 1;
    }
    return seq;
}

public static void main(String[] args) {
    int[] numeri = creaSequenza(5);
    // numeri = {1, 2, 3, 4, 5}
}
```

# ARRAYS.TOSTRING

La classe `java.util.Arrays` offre metodi utili per lavorare con gli array. Il primo: stampare un array leggibile

```
import java.util.Arrays;

int[] voti = {7, 8, 6, 9, 5};
System.out.println(Arrays.toString(voti));
// → [7, 8, 6, 9, 5]
```

- *Serve l'* `import java.util.Arrays` *in cima al file*
- *Restituisce una stringa con i valori tra parentesi quadre*
- *Molto utile per il debugging*

# ARRAYS.SORT

`Arrays.sort` ordina l'array in ordine crescente, **modificando l'originale**

```
int[] numeri = {5, 3, 8, 1, 2};
Arrays.sort(numeri);
System.out.println(Arrays.toString(numeri));
// → [1, 2, 3, 5, 8]
```

- *L'ordinamento è **in-place**: modifica l'array, non ne crea uno nuovo*
- *Funziona con `int[]`, `double[]`, `String[]` e altri tipi*
- *Le stringhe si ordinano per codice dei caratteri: le maiuscole vengono prima delle minuscole*

# ARRAYS.COPYOF

`Arrays.copyOf` crea una **copia indipendente** dell'array

```
int[] originale = {7, 8, 6, 9, 5};
int[] copia = Arrays.copyOf(originale, originale.length);

copia[0] = 99;
System.out.println(Arrays.toString(originale));
// → [7, 8, 6, 9, 5]  (invariato)
System.out.println(Arrays.toString(copia));
// → [99, 8, 6, 9, 5]
```

- *Il secondo parametro è la lunghezza della copia*
- *Se la lunghezza è maggiore, le celle extra prendono il valore predefinito:* `0`, `false` *o* `null`
- *Utile per non modificare l'array originale*

# JAVA.UTIL.ARRAYS

I metodi più utili della classe `Arrays`

| Metodo | Cosa fa |
|---|---|
| `Arrays.toString(a)` | Restituisce una stringa leggibile dell'array |
| `Arrays.sort(a)` | Ordina l'array in ordine crescente (in-place) |
| `Arrays.copyOf(a, n)` | Crea una copia di lunghezza `n` |
| `Arrays.fill(a, val)` | Riempie tutte le celle con `val` |
| `Arrays.equals(a, b)` | Confronta due array elemento per elemento |
| `Arrays.binarySearch(a, key)` | Cerca `key` in un array ordinato |

Tutti richiedono `import java.util.Arrays`

# ATTENZIONE: == CON GLI ARRAY

Come per le stringhe, `==` non confronta il contenuto ma il **riferimento**

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};

System.out.println(a == b);                // false
System.out.println(Arrays.equals(a, b)); // true
```

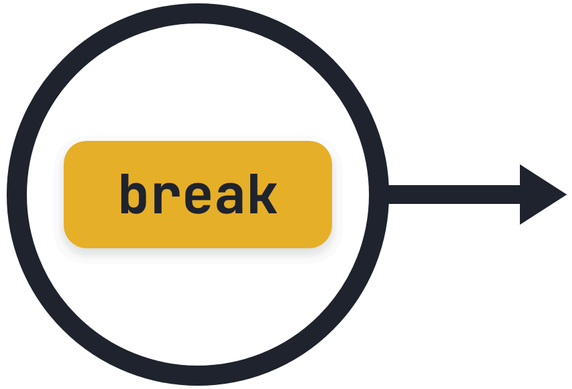
- `==` *verifica se* `a` *e* `b` *puntano allo* ***stesso oggetto***
- `Arrays.equals` *confronta i* ***valori*** *elemento per elemento*
- *Stesso principio di* `.equals()` *per le stringhe*

# CONFRONTO CON PYTHON

In Python le liste sono più flessibili, in Java gli array sono più rigidi

| Caratteristica | Java `int[]` | Python `list` |
|---|---|---|
| Dimensione | Fissa | Dinamica |
| Tipo degli elementi | Unico | Misto |
| Aggiungere elementi | Non possibile | `append()` |
| Indice negativo | Eccezione | `lista[-1]` |
| Stampare | `Arrays.toString()` | `print(lista)` |
| Ordinare | `Arrays.sort()` | `lista.sort()` |

# 04
# ARRAY BIDIMENSIONALI



# ARRAY 2D: CREARE

Un array bidimensionale è un **array di array**: una griglia con righe e colonne

```
// 3 righe, 4 colonne
int[][] griglia = new int[3][4];
// con valori espliciti: 2 righe, 3 colonne
int[][] matrice = {{1, 2, 3}, {4, 5, 6}};
```

- `int[][]` — *due coppie di quadre indicano due dimensioni*
- `griglia.length` — *numero di righe*
- `griglia[0].length` — *numero di colonne*

# ARRAY 2D: ACCEDERE

Si usano due indici: il primo per la **riga**, il secondo per la **colonna**

```
int[][] matrice = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

System.out.println(matrice[0][0]);  // 1 (riga 0, colonna 0)
System.out.println(matrice[1][2]);  // 6 (riga 1, colonna 2)
System.out.println(matrice[2][1]);  // 8 (riga 2, colonna 1)

matrice[1][1] = 99;                 // modifica la cella centrale
```

# ITERARE CON DUE FOR

Per visitare tutti gli elementi si usano due cicli annidati: uno per le righe, uno per le colonne

```java
int[][] matrice = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

for (int r = 0; r < matrice.length; r++) {
    for (int c = 0; c < matrice[r].length; c++) {
        System.out.print(matrice[r][c] + "  ");
    }
    System.out.println();
}
```

Terminale

```
1  2  3
4  5  6
7  8  9
```

# ESEMPIO: LA GRIGLIA

Gli array 2D rappresentano bene le griglie: scacchiere, campi di gioco

```java
// Griglia di tris 3x3
char[][] tris = {
    {'X', ' ', 'O'},
    {' ', 'X', ' '},
    {'O', ' ', 'X'}
};
for (int r = 0; r < tris.length; r++) {
    for (int c = 0; c < tris[r].length; c++) {
        System.out.print(tris[r][c]);
        if (c < tris[r].length - 1) System.out.print(" | ");
    }
    System.out.println();
}
```



# ESEMPIO: BATTAGLIA NAVALE

Una griglia 5x5 può rappresentare il campo di gioco. Ogni cella ha un valore che indica il suo stato

```
// legenda: '~' acqua, 'N' nave, 'X' colpito, 'O' mancato
char[][] campo = new char[5][5];

// inizializzare tutto ad acqua
for (int r = 0; r < campo.length; r++) {
    for (int c = 0; c < campo[r].length; c++) {
        campo[r][c] = '~';
    }
}

// piazzare una nave (3 celle, orizzontale, riga 2)
campo[2][1] = 'N';
campo[2][2] = 'N';
campo[2][3] = 'N';
```

- *Array 2D di* `char` *— ogni cella è un carattere*

# ERRORI COMUNI CON GLI ARRAY

I bug più frequenti e come evitarli

| Errore | Causa | Correzione |
|---|---|---|
| `ArrayIndexOutOfBoundsException` | `<=` invece di `<` | Usare `i < arr.length` |
| Errore di compilazione: `might not have been initialized` | `int[] a;` senza `new` | Aggiungere `= new int[n]` |
| Stampa codice hash | `println(arr)` | Usare `Arrays.toString()` |
| `==` tra array | Confronta i riferimenti | Usare `Arrays.equals()` |
| Copia con `=` | Copia il riferimento, non i dati | Usare `Arrays.copyOf()` |
| Modificare nel for-each | `v` è una copia locale | Usare il for classico |

# COPIA: RIFERIMENTO VS DATI

Con `=` si copia il riferimento: entrambe le variabili puntano allo stesso array

```
// copia del RIFERIMENTO (stesso array!)
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 99;
System.out.println(a[0]);  // 99 — anche a è cambiato!

// copia dei DATI (array indipendente)
int[] a = {1, 2, 3};
int[] b = Arrays.copyOf(a, a.length);
b[0] = 99;
System.out.println(a[0]);  // 1 — a non cambia
```

# QUIZ

Cosa stampa il seguente frammento?

```
int[][] m = {
    {10, 20},
    {30, 40},
    {50, 60}
};

System.out.println(m.length);
System.out.println(m[0].length);
System.out.println(m[2][1]);

int[] riga = m[1];
riga[0] = 99;
System.out.println(m[1][0]);
```

# RISPOSTE

- `m.length` — *numero di righe:* ***3***
- `m[0].length` — *colonne della prima riga:* ***2***
- `m[2][1]` — *riga 2, colonna 1:* ***60***
- `m[1][0]` *dopo la modifica:* ***99*** *— perché* `riga` *è un riferimento alla stessa riga dell'array, non una copia*

# RIEPILOGO

- ***Array*** — *sequenza di lunghezza fissa, indice da 0, stesso tipo*
- ***Creazione*** — `new int[n]` *oppure* `{v1, v2, ...}`
- ***Iterazione*** — `for` *classico (con indice) o* `for-each` *(senza)*
- ***Riferimento*** — *il metodo riceve il riferimento e lavora sullo stesso array*
- ***java.util.Arrays*** — `toString`, `sort`, `copyOf`, `equals`
- ***Array 2D*** — `int[][]`, *due indici, due cicli annidati*
- ***Trappole*** — `<=` *nel for,* `==` *tra array,* `=` *per copiare*

# DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.0